RIPHAH INTERNATIONAL UNIVERSITY

MINI RPG ADVENTURE GAME

Data Structures Project

Submitted By:

Name: Zunaira Aslam

Sap ID: 69748

Semester: 3rd Semester

Program: BS Computer Science (BSCS)

Submitted To:

Teacher Name: Ma'am Hareem Haider

Submission Date:

28 April 2026

1. Project Title

Design and Implementation of an Algorithm-Based System Using Data Structures

Project Name: Mini RPG Adventure Game

2. Problem Description

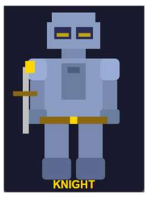
Mini RPG Adventure Game is a console-based game written in C++. The player selects a hero class, fights enemies one by one, buys items from a shop, and can undo previous actions.

The project is built using core Data Structures and Algorithms studied in the Data Structures course. It uses:

- **Stack** for Undo System
- **Queue** for Enemy Management
- **Binary Search** for fast item searching in shop
- **Linear Search** for normal item searching in shop
- **Recursion** for countdown before battle

The game demonstrates practical implementation of these concepts through a simple RPG simulation.

How the Game Works

1. The player chooses a hero:
    - Knight (HP: 120, ATK: 15) 

    - Archer (HP: 90, ATK: 25) 

    - Mage (HP: 80, ATK: 30) 

2. Three enemies are loaded into a Queue:
    - Goblin 

    - Orc 

    - Dragon 

3. In every turn, the player can:
    - Attack enemy
    - Visit Shop
    - Undo previous action
4. In the Shop, the player can choose:
    - Binary Search
    - Linear Search

to find required items

5. Before every fight or shop purchase, the game saves the current state using Stack
6. Undo restores:
    - Previous HP
    - Previous Gold

    Note: Attack (ATK) is not restored in the current version because only HP and Gold are stored in Stack.
7. The game ends when:
    - All enemies are defeated → **YOU WIN**
    - Player HP becomes 0 → **GAME OVER**

3. Data Structures Used

3.1 Stack — Undo / Action History System

A custom singly linked-list based Stack stores the player's previous game state before every important action.

It saves:

- Action name
- HP
- Gold

This allows the player to undo mistakes such as bad fights or wrong purchases.

Node Structure

```
struct StackNode {
    string action;
    int hp, gold;
    StackNode* next;
};
```

Operations Implemented

push(action, hp, gold)

Called before every fight and shop purchase. Stores current state at top of stack.

pop(action, hp, gold)

Called during Undo. Restores previous HP and Gold.

display()

Prints complete action history.

Console Visualization

```
>> Progress saved!
[History]: Fight -> Buy -> NULL
Undo successful!
```

3.2 Queue — Enemy Spawn System

A custom singly linked-list based Queue manages enemy order.

Enemies are inserted at the rear and fought from the front. When an enemy is defeated, it is removed from the queue.

This creates a proper enemy progression system.

Node Structure

```
struct Enemy {
    string name;
    int hp, dmg, reward;
    Enemy* next;
};
```

Operations Implemented

enqueue(name, hp, dmg, reward)

Adds enemy to rear of queue.

dequeue()

Removes defeated enemy from front.

getFront()

Returns current enemy without removing.

isEmpty()

Checks whether queue is empty.

display()

Prints all remaining enemies.

Console Visualization

```
[Enemies]: Goblin(HP:30) Orc(HP:50) Dragon(HP:80)
Goblin defeated!
[Enemies]: Orc(HP:50) Dragon(HP:80)
```

4. Algorithms Implemented

4.1 Binary Search — Shop Item Lookup

The shop uses a sorted array:

```
{Elixir, Potion, Shield, Sword}
```

When Binary Search is selected, the algorithm repeatedly divides the search range in half until the required item is found.

This is faster than Linear Search.

Console Visualization

```
[Binary Search]
Checking: Potion
```

```
Checking: Shield
Checking: Sword
```

4.2 Linear Search — Shop Item Lookup

When Linear Search is selected, the algorithm checks items one by one from the beginning until the item is found.

This is slower than Binary Search but works on both sorted and unsorted arrays.

Console Visualization

```
[Linear Search]
Checking: Elixir
Checking: Potion
Checking: Shield
```

5. Complexity Analysis

5.1 Stack

Operation	Best Case	Average Case	Worst Case	Space
push()	O(1)	O(1)	O(1)	O(n)
pop()	O(1)	O(1)	O(1)	O(1)
display()	O(n)	O(n)	O(n)	O(1)

5.2 Queue

Operation	Best Case	Average Case	Worst Case	Space
enqueue()	O(1)	O(1)	O(1)	O(n)
dequeue()	O(1)	O(1)	O(1)	O(1)
display()	O(n)	O(n)	O(n)	O(1)

5.3 Binary Search

Case	Time Complexity	Explanation
Best Case	O(1)	Item found at middle index immediately
Average Case	O(log n)	Search space keeps dividing
Worst Case	O(log n)	Item is last possible check or not present
Space	O(1)	No extra memory used

5.4 Linear Search

Case	Time Complexity	Explanation
Best Case	O(1)	First item matches
Average Case	O(n)	Item found after several checks
Worst Case	O(n)	Last item or item not found
Space	O(1)	No extra memory used

5.5 Binary Search vs Linear Search

Feature	Binary Search	Linear Search
Best Case	O(1)	O(1)
Worst Case	O(log n)	O(n)
Requirement	Sorted array required	Works on any array
Used In	Shop (Choice 1)	Shop (Choice 2)

5.6 Recursion Complexity

Case	**Complexity**	**Explanation**
Time Complexity	O(n)	Function calls itself n times
Space Complexity	O(n)	Recursive call stack stores n function calls

Additional Note About HP

For simplicity, maximum HP is capped at **120 for all heroes** during healing.

This means even Archer and Mage can heal up to 120 HP when buying Potion or Elixir.

```
if (hp > 120)
    hp = 120;
```

Final Result

This project successfully fulfills all requirements:

- Two Data Structures implemented
- Two Searching Algorithms implemented
- Sorting algorithm implemented
- Complexity Analysis included

- Console-based Visualization provided

The Mini RPG Adventure Game demonstrates how Data Structures can be applied in real-world logical systems through a simple and interactive game.

←----------→